partnr

Build a Secure AWS Payments API Infrastructure with Terraform and LocalStack

Back

Mandatory

Domain

Cloud Computing | DevOps

Skills

Cloud Computing | Compliance | Encryption | Infrastructure as Code (IaC) | Management | Serverless | Local Development Workflows

Difficulty

Tools

Report Issue

partnr

1. Project Introduction and Core Concepts

Welcome to a hands-on project designed to simulate the foundational work of a cloud engineer at a modern fintech startup. Your mission is to build the core infrastructure for a new payments API, but with a critical constraint: everything must be defined as code and thoroughly tested in a local environment before it's approved for a real cloud deployment. This practice, known as Infrastructure as Code (IaC), is central to modern DevOps and cloud engineering. It ensures that your infrastructure is repeatable, version-controlled, and less prone to human error.

1.1. Why Infrastructure as Code (IaC)?

Infrastructure as Code is the process of managing and provisioning computer data centers through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools. For this project, we'll use Terraform, one of the most popular open-source IaC tools.

1. **What it solves:** IaC eliminates the manual, error-prone process of setting up servers, databases, and networking. Manually configured environments often suffer from "configuration drift," where undocumented changes make the production environment different from staging or development. This can lead to bugs that are hard to reproduce and diagnose.
2. **Relevance to this task:** By writing our AWS infrastructure in Terraform, we create a single source of truth. Anyone can run `terraform apply` and get the exact same environment. This is crucial for a fintech company that requires auditable and consistent infrastructure to meet compliance standards.
3. **Key Takeaway:** Think of your `.tf` files as the blueprint for your entire cloud setup.

1.2. The Power of Local Cloud Emulation with LocalStack

Constantly deploying to a real AWS account during development can be slow and expensive. Every change requires a round trip to AWS, and even small test resources can incur costs. LocalStack solves this by providing a fully functional local AWS cloud stack that you can run in a Docker container.

1. **What it solves:** It provides a rapid feedback loop for developers. You can deploy and destroy your entire cloud infrastructure on your own machine in seconds, without an internet connection and at zero cost. It's the perfect sandbox for testing your Terraform code.
2. **Relevance to this task:** We will run LocalStack via Docker Compose. All our `terraform apply` commands will be directed at this local instance, allowing us to verify that our S3 buckets, DynamoDB tables, and IAM policies are configured correctly before we ever touch a real AWS account. This is the "validate locally first" principle in action.

3. **Common Pitfall:** A frequent mistake is forgetting that LocalStack is an emulator. While it

partnr

1.3. Security First: A Fintech Imperative

For any company handling financial data, security is not an afterthought—it's the foundation. We will focus on three core security principles:

1. **Encryption at Rest:** All data stored in our S3 bucket and DynamoDB table must be encrypted. We'll use AWS Key Management Service (KMS) with a customer-managed key (CMK) to achieve this. This gives us granular control over who can access the key to encrypt or decrypt data.
2. **Principle of Least Privilege (PoLP):** Our Lambda function will need permission to interact with other AWS services. We will craft an IAM policy that grants it *only* the specific permissions it needs to do its job, and nothing more. We will strictly avoid using wildcards (`*`) in our IAM policies, which is a common but dangerous shortcut.
3. **No Public Data:** Financial data should never be publicly accessible. We will configure our S3 bucket to explicitly block all public access, a critical safeguard against accidental data exposure.

2. System Architecture and Design

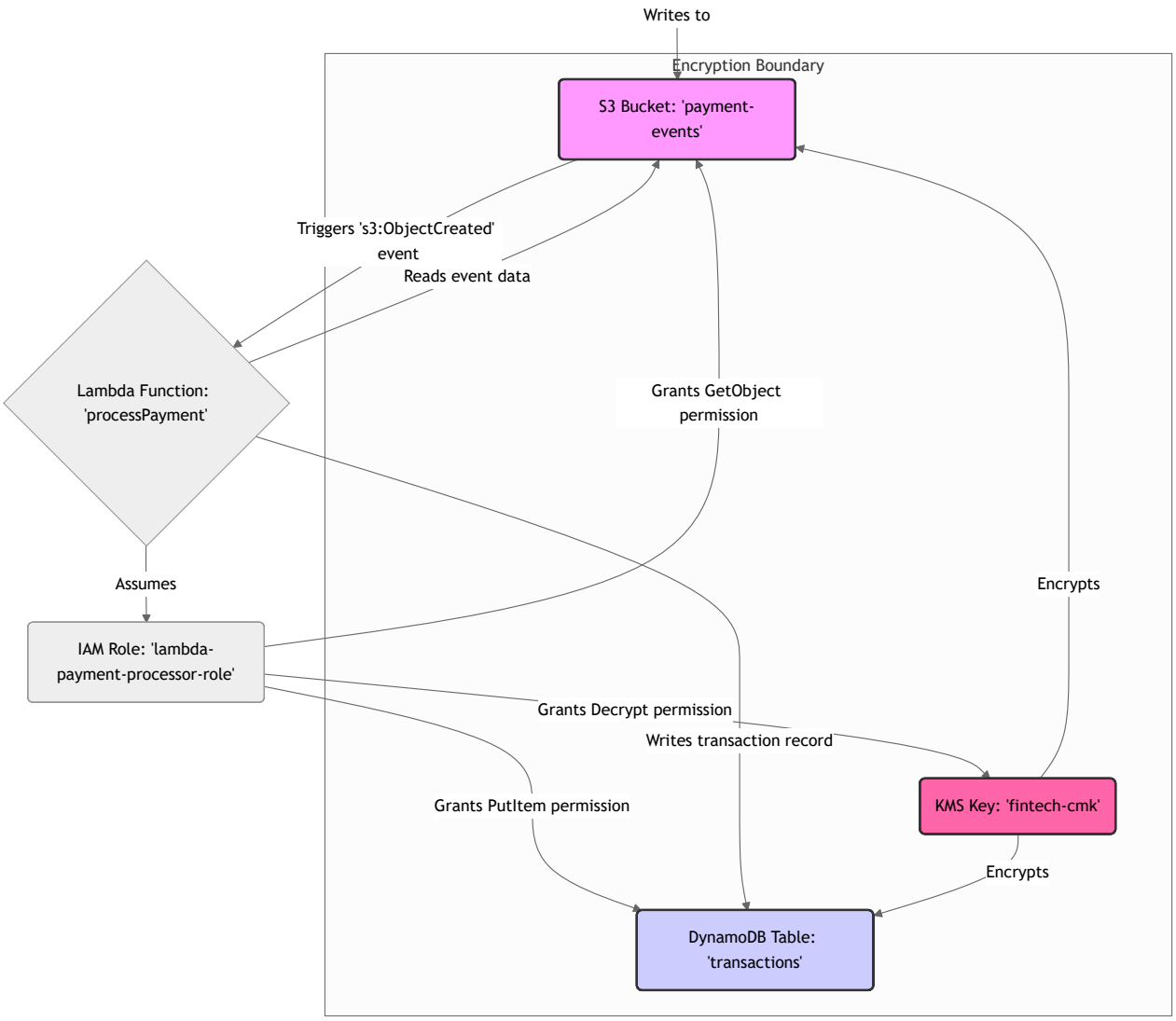
Before writing any code, let's visualize the system we are building. The architecture is an event-driven, serverless workflow, which is highly scalable and cost-effective. A new payment event (represented by a file) will trigger a series of automated actions.

2.1. Architectural Diagram

Here is a diagram of the components and their interactions. This is the blueprint for our Terraform implementation.

Expand

partnr



Data Flow Explained:

1. **Initiation:** A client application (which we will simulate with the AWS CLI) uploads a JSON file representing a payment event to a dedicated S3 bucket.
2. **Trigger:** The S3 bucket is configured to send an `s3:ObjectCreated` event notification whenever a new object is created. This event is directed to our Lambda function.
3. **Execution:** The Lambda function is invoked, receiving the event payload which contains information about the S3 object (bucket name and object key).
4. **Authorization:** The Lambda function assumes its assigned IAM Role. This role grants it temporary credentials with precisely defined permissions.
5. **Processing:** Using these credentials, the function's code will:
    1. Call the S3 `GetObject` API to retrieve the payment event file.
    2. Parse the file's contents.
    3. Call the DynamoDB `PutItem` API to write a new record into the `transactions` table.

6. **Encryption:** Both the S3 bucket and the DynamoDB table are encrypted using our

partnr

listed as a user in the key's policy.

3. Step-by-Step Implementation Guide

This section will walk you through creating the project from scratch. Follow the steps carefully.

3.1. Setting Up Your Local Development Environment

First, we need to set up Docker, LocalStack, Terraform, and the AWS CLI.

1. **Install Prerequisites:** Make sure you have Docker, Docker Desktop, Terraform, and the AWS CLI installed on your system.

2. **Create Project Structure:** Create a root directory for your project. Inside it, create the following structure:

```
fintech-infra/
├── src/                 # For Lambda function code
│   └── process_payment.py
├── docker-compose.yml   # For LocalStack
├── main.tf              # Main Terraform configuration
├── variables.tf         # Terraform variables
├── outputs.tf           # Terraform outputs
├── compliance_check.sh  # Our compliance script
├── .env.example         # Example environment variables
└── .gitignore           # Git ignore file
```

3. **Configure LocalStack:** In `docker-compose.yml`, define the LocalStack service. This file tells Docker how to run LocalStack and which AWS services to enable.

```yaml
# docker-compose.yml
version: '3.8'

services:
  localstack:
    container_name: "localstack_fintech"
    image: localstack/localstack:latest
    ports:
      - "4566:4566"            # Main endpoint for all services
    environment:
      - SERVICES=s3,dynamodb,iam,kms,lambda # Specify services to enable
      - DEBUG=1
      - DATA_DIR=/tmp/localstack/data
    volumes:
      - "./localstack_data:/tmp/localstack"
      - "/var/run/docker.sock:/var/run/docker.sock"
```

4. **Start LocalStack:** Navigate to your project root in the terminal and run `docker-compose up -d`.

partnr

5. **Configure AWS CLI for LocalStack:** To make the AWS CLI talk to your local container instead of the real AWS, you need to configure its endpoint. You can do this by setting environment variables or creating a dedicated profile. For simplicity, we'll use environment variables. Create a `.env` file from `.env.example` and add:

```
# .env.example
AWS_ACCESS_KEY_ID=test
AWS_SECRET_ACCESS_KEY=test
AWS_DEFAULT_REGION=us-east-1
# This is the key part - it points all AWS CLI/SDK calls to our container
AWS_ENDPOINT_URL=http://localhost:4566
```

Source this file in your terminal: `source .env`. Now, any `aws` command you run will target LocalStack. You can test this with `aws s3 ls`. It should return an empty list.

3.2. Writing the Terraform Configuration

Now we'll define our resources in Terraform. We will be using the official AWS provider.

1. **Provider Configuration:** In `main.tf`, configure the AWS provider to talk to LocalStack. This is similar to how we configured the CLI.

```
# main.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.0"
    }
  }
}

provider "aws" {
  region                      = "us-east-1"
  access_key                  = "test"
  secret_key                  = "test"
  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true

  # Point provider to LocalStack endpoints
  endpoints {
    s3         = "http://localhost:4566"
    dynamodb   = "http://localhost:4566"
    iam        = "http://localhost:4566"
    kms        = "http://localhost:4566"
    lambda     = "http://localhost:4566"
```

```
}
```

partnr

2. **Define the KMS Key:** This is our cornerstone for encryption.

```
# main.tf (continued)
resource "aws_kms_key" "fintech_key" {
  description             = "KMS key for fintech app data encryption"
  deletio-window_in_days = 7
  enable_key_rotation    = true
}
```

3. **Define the S3 Bucket:** Notice the explicit security configurations.

```
# main.tf (continued)
resource "aws_s3_bucket" "payment_events" {
  bucket = "fintech-payment-events-${terraform.workspace}"
}

resource "aws_s3_bucket_versioning" "versioning_example" {
  bucket = aws_s3_bucket.payment_events.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "encryption_config" {
  bucket = aws_s3_bucket.payment_events.id
  rule {
    apply_server_side_encryption_by_default {
      kms_master_key_id = aws_kms_key.fintech_key.arn
      sse_algorithm     = "aws:kms"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "pab" {
  bucket                  = aws_s3_bucket.payment_events.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

4. **Define the DynamoDB Table:** Again, we specify our KMS key for encryption.

```
# main.tf (continued)
resource "aws_dynamodb_table" "transactions" {
  name           = "transactions-${terraform.workspace}"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "TransactionID"
```

partnr

```
  }

  server_side_encryption {
    enabled     = true
    kms_key_arn = aws_kms_key.fintech_key.arn
  }
}
```

5. **Define the IAM Role and Least-Privilege Policy:** This is the most security-sensitive part. We create a role that the Lambda service can assume, then attach a policy with very specific, non-wildcard permissions.

```
# main.tf (continued)
data "aws_iam_policy_document" "lambda_assume_role_policy" {
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type        = "Service"
      identifiers = ["lambda.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "lambda_payment_processor_role" {
  name               = "lambda-payment-processor-role-${terraform.workspace}"
  assume_role_policy = data.aws_iam_policy_document.lambda_assume_role_policy.json
}

data "aws_iam_policy_document" "lambda_permissions" {
  # CloudWatch Logs permissions
  statement {
    actions = [
      "logs:CreateLogGroup",
      "logs:CreateLogStream",
      "logs:PutLogEvents"
    ]
    resources = ["arn:aws:logs:*:*:*"] # This is one of the few acceptable wildcards
  }

  # S3 read permission for the specific bucket
  statement {
    actions   = ["s3:GetObject"]
    resources = ["${aws_s3_bucket.payment_events.arn}/*"] # Note the /* for objects
  }

  # DynamoDB write permission for the specific table
  statement {
    actions   = ["dynamodb:PutItem"]
    resources = [aws_dynamodb_table.transactions.arn]
  }
```

```
}
```

partnr

```
  policy = data.aws_iam_policy_document.lambda_permissions.json
}

resource "aws_iam_role_policy_attachment" "attach_lambda_policy" {
  role       = aws_iam_role.lambda_payment_processor_role.name
  policy_arn = aws_iam_policy.lambda_policy.arn
}
```

6. **Define the Lambda Function and Trigger:** Finally, we define the Lambda function itself and connect it to the S3 bucket trigger. First, create a placeholder Lambda function in `src/process_payment.py`:

```
# src/process_payment.py
import json
import boto3
import os
from datetime import datetime

# Note: In LocalStack, endpoint_url must be specified for clients
endpoint_url = f"http://{os.environ.get('LOCALSTACK_HOSTNAME')}:{os.environ.get('EDGE_PORT')}
dynamodb = boto3.resource('dynamodb', endpoint_url=endpoint_url)
table = dynamodb.Table(os.environ['DYNAMODB_TABLE_NAME'])

def handler(event, context):
    print(json.dumps(event))
    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = record['s3']['object']['key']

        # Create a transaction ID from the key and current time
        transaction_id = f"{key}-{datetime.utcnow().isoformat()}"

        # Write to DynamoDB
        table.put_item(
            Item={
                'TransactionID': transaction_id,
                'Bucket': bucket,
                'ObjectKey': key,
                'Status': 'PROCESSED'
            }
        )
        print(f"Processed {key} from {bucket}. TransactionID: {transaction_id}")
    return {'status': 'success'}
```

Now, add the Terraform resources to deploy it. We'll use a `data` source to zip the code on the fly.

partnr

```
  source_file = "${path.module}/src/process_payment.py"
  output_path = "${path.module}/dist/process_payment.zip"
}

resource "aws_lambda_function" "process_payment_lambda" {
  function_name = "process-payment-${terraform.workspace}"
  handler       = "process_payment.handler"
  runtime       = "python3.9"
  role          = aws_iam_role.lambda_payment_processor_role.arn

  filename         = data.archive_file.lambda_zip.output_path
  source_code_hash = data.archive_file.lambda_zip.output_base64sha256

  environment {
    variables = {
      DYNAMODB_TABLE_NAME = aws_dynamodb_table.transactions.name
    }
  }
}

resource "aws_s3_bucket_notification" "bucket_notification" {
  bucket = aws_s3_bucket.payment_events.id

  lambda_function {
    lambda_function_arn = aws_lambda_function.process_payment_lambda.arn
    events              = ["s3:ObjectCreated:*"]
  }

  depends_on = [aws_lambda_permission.allow_s3]
}

resource "aws_lambda_permission" "allow_s3" {
  statement_id  = "AllowS3Invoke"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.process_payment_lambda.function_name
  principal     = "s3.amazonaws.com"
  source_arn    = aws_s3_bucket.payment_events.arn
}
```

3.3. Creating the Compliance Script

Now, let's write our `compliance_check.sh` script. This script will use the AWS CLI to inspect the deployed resources and verify they meet our security standards. Make it executable with `chmod +x compliance_check.sh`.

```
#!/bin/bash
# compliance_check.sh

# Fail script on any error
set -e
```

partnr

```
fi

WORKSPACE=${1:-dev} # Default to 'dev' workspace if no argument is provided
BUCKET_NAME="fintech-payment-events-$WORKSPACE"
TABLE_NAME="transactions-$WORKSPACE"
KEY_ARN=$(aws --endpoint-url=$AWS_ENDPOINT_URL kms list-aliases | jq -r '.Aliases[] | select(.Alia
POLICY_ARN=$(aws --endpoint-url=$AWS_ENDPOINT_URL iam list-policies --scope Local | jq -r --arg ws

echo "--- Running Compliance Checks for workspace: $WORKSPACE ---"

# 1. Check S3 Public Access Block
echo "[CHECK 1] Verifying S3 Public Access Block on bucket: $BUCKET_NAME..."
PUBLIC_ACCESS_BLOCK=$(aws --endpoint-url=$AWS_ENDPOINT_URL s3api get-public-access-block --bucket 
if [[ $(echo $PUBLIC_ACCESS_BLOCK | jq '.PublicAccessBlockConfiguration.BlockPublicAcls') == "true
      $(echo $PUBLIC_ACCESS_BLOCK | jq '.PublicAccessBlockConfiguration.BlockPublicPolicy') == "tr
      $(echo $PUBLIC_ACCESS_BLOCK | jq '.PublicAccessBlockConfiguration.IgnorePublicAcls') == "tru
      $(echo $PUBLIC_ACCESS_BLOCK | jq '.PublicAccessBlockConfiguration.RestrictPublicBuckets') ==
    echo "  ✓ SUCCESS: S3 bucket has public access block fully enabled."
else
    echo "  ✗ FAILURE: S3 public access block is not correctly configured."
    exit 1
fi

# 2. Check S3 Encryption
echo "[CHECK 2] Verifying S3 Encryption on bucket: $BUCKET_NAME..."
BUCKET_ENCRYPTION=$(aws --endpoint-url=$AWS_ENDPOINT_URL s3api get-bucket-encryption --bucket $BUC
SSE_ALGORITHM=$(echo $BUCKET_ENCRYPTION | jq -r '.ServerSideEncryptionConfiguration.Rules[0].Apply
if [[ $SSE_ALGORITHM == "aws:kms" ]]; then
    echo "  ✓ SUCCESS: S3 bucket is encrypted with aws:kms."
else
    echo "  ✗ FAILURE: S3 bucket encryption is not aws:kms. Found: $SSE_ALGORITHM"
    exit 1
fi

# 3. Check DynamoDB Encryption
echo "[CHECK 3] Verifying DynamoDB Encryption on table: $TABLE_NAME..."
TABLE_DESCRIPTION=$(aws --endpoint-url=$AWS_ENDPOINT_URL dynamodb describe-table --table-name $TAB
SSE_STATUS=$(echo $TABLE_DESCRIPTION | jq -r '.Table.SSEDescription.Status')
if [[ $SSE_STATUS == "ENABLED" ]]; then
    echo "  ✓ SUCCESS: DynamoDB table encryption is enabled."
else
    echo "  ✗ FAILURE: DynamoDB encryption is not enabled. Status: $SSE_STATUS"
    exit 1
fi

# 4. Check IAM Policy for Wildcard Actions
echo "[CHECK 4] Verifying IAM policy for wildcard actions..."
POLICY_VERSION=$(aws --endpoint-url=$AWS_ENDPOINT_URL iam get-policy --policy-arn $POLICY_ARN | jq
POLICY_DOC=$(aws --endpoint-url=$AWS_ENDPOINT_URL iam get-policy-version --policy-arn $POLICY_ARN 
WILDCARD_ACTIONS=$(echo $POLICY_DOC | jq -r '.PolicyVersion.Document.Statement[].Action | to_a | .

# Filter out the acceptable CloudWatch wildcard
```

```
ALLOWED_WILDCARD="logs:*"
```

partnr

```
    echo "  ✓ SUCCESS: No forbidden wildcard actions found in IAM policy."
else
    echo "  ✗ FAILURE: Forbidden wildcard actions found: $FILTERED_WILDCARDS"
    exit 1
fi

echo "--- All Compliance Checks Passed! ---"
```

3.4. Deployment and Verification

Now, let's bring it all together.

1. **Initialize Terraform:** Run `terraform init` in your terminal. This downloads the AWS provider.
2. **Create Workspaces:** We need separate environments. Run:
    1. `terraform workspace new dev`
    2. `terraform workspace new staging`
    3. Switch back to dev: `terraform workspace select dev`
3. **Apply Configuration:** Run `terraform apply -auto-approve`. Terraform will now provision all the resources inside your LocalStack container.
4. **Run Compliance Check:** Execute your script: `./compliance_check.sh dev`. It should run through all the checks and report success.
5. **Test the E2E Flow:**
    1. Create a dummy event file: `echo '{"paymentId": "123", "amount": 100}' > payment.json`
    2. Upload it to S3: `aws s3 cp payment.json s3://fintech-payment-events-dev/`
    3. Check Lambda logs (in the `docker-compose up` terminal) to see the function being invoked.
    4. Verify the record in DynamoDB: `aws dynamodb scan --table-name transactions-dev`. You should see a new item corresponding to your S3 upload.

4. FAQ and Common Issues

1. **Q: My `terraform apply` fails with a network error.**
    1. **A:** Ensure your LocalStack container is running (`docker ps`). Also, verify that your AWS provider configuration in `main.tf` and your environment variables correctly point to `http://localhost:4566`.
2. **Q: The Lambda function isn't triggered after uploading a file to S3.**
    1. **A:** This is often a permissions issue. Double-check the `aws_lambda_permission` resource, which allows S3 to invoke the function. Also, inspect the `aws_s3_bucket_notification` to ensure it's configured for the correct event type

(`s3:ObjectCreated:*`) and points to the correct Lambda ARN. LocalStack logs can also

partnr

1. **A:** Workspaces provide a clean separation of state. Each workspace gets its own `terraform.tfstate` file, meaning the state of your `dev` environment is completely isolated from `staging` . This prevents accidental changes to one environment when working on another. It's the standard way to manage multiple environments with Terraform.

4. **Q: Why not use an `s3:GetObject` wildcard like `*` in the IAM policy?**

   1. **A:** The Principle of Least Privilege dictates that a component should only have access to the resources it absolutely requires. While `s3:*` is easier to write, it would allow our Lambda to *delete* objects, change bucket policies, etc. By specifying only `s3:GetObject` , we limit the potential damage if the function's code is ever compromised.

Core Requirements

1. A `docker-compose.yml` file must be present in the root directory to configure and launch the LocalStack environment. The composition must start successfully using `docker-compose up` .

   Contract Specification

   File Location: `docker-compose.yml`

   Required Services:

   1. `localstack` : The service running the `localstack/localstack` image.

   Required Configuration:

   1. The container must expose port `4566` to the host.
   2. The `SERVICES` environment variable must be set and include at least `s3` , `dynamodb` , `iam` , `kms` , and `lambda` .

   Verification:

   1. The `docker-compose.yml` file must exist at the project root.
   2. The command `docker-compose up -d` must execute without errors.
   3. After startup, `docker ps` must show a container for `localstack_fintech` in a running state.
   4. The command `curl http://localhost:4566/health` should return a JSON response indicating the required services are `available` or `running` .

---

2. The Terraform configuration must define a customer-managed AWS KMS Key that will be

partnr

Contract Specification

Terraform Resource: `aws_kms_key`

Required Properties:

1. `enable_key_rotation` must be set to `true`.

Verification:

1. Run `terraform apply` to provision the infrastructure.
2. Use the AWS CLI against the LocalStack endpoint to list KMS keys: `aws kms list-keys`.
3. The output must contain at least one customer-managed key.
4. Describe the key using its ID and verify that `KeyManager` is `CUSTOMER` and `KeySpec` is `SYMMETRIC_DEFAULT`.

---

3. The Terraform configuration must define an S3 bucket with versioning, server-side encryption using the custom KMS key, and a public access block.

Contract Specification

Terraform Resources:

1. `aws_s3_bucket`
2. `aws_s3_bucket_versioning`
3. `aws_s3_bucket_server_side_encryption_configuration`
4. `aws_s3_bucket_public_access_block`

Verification:

1. Get the bucket name from the Terraform state.
2. **Check Versioning:** Run `aws s3api get-bucket-versioning --bucket <bucket-name>`. The `Status` must be `Enabled`.
3. **Check Encryption:** Run `aws s3api get-bucket-encryption --bucket <bucket-name>`. The output must show `SSEAlgorithm` as `aws:kms` and the `KMSMasterKeyID` must match the ARN of the created KMS key.
4. **Check Public Access Block:** Run `aws s3api get-public-access-block --bucket <bucket-name>`. The `PublicAccessBlockConfiguration` must have `BlockPublicAcls`, `BlockPublicPolicy`, `IgnorePublicAcls`, and `RestrictPublicBuckets` all set to `true`.

---

4. The Terraform configuration must define a DynamoDB table with server-side encryption enabled, using the custom KMS key.

Contract Specification

partnr

Required Properties:

1. The `server_side_encryption` block must be present and `enabled` set to `true`.
2. The `kms_key_arn` within the encryption block must be set to the ARN of the created KMS key.

Verification:

1. Get the table name from the Terraform state.
2. Run `aws dynamodb describe-table --table-name <table-name>`.
3. The response JSON must contain an `SSEDescription` object.
4. Within `SSEDescription`, the `Status` must be `ENABLED` and the `KMSMasterKeyArn` must match the ARN of the created KMS key.

---

5. The Terraform configuration must define an IAM Role and an attached IAM Policy for the Lambda function that adheres to the principle of least privilege. No wildcard `*` actions are permitted, with the exception of necessary `logs:*` permissions.

Contract Specification

Terraform Resources:

1. `aws_iam_role` with an `assume_role_policy` for `lambda.amazonaws.com`.
2. `aws_iam_policy` or `data "aws_iam_policy_document"` defining permissions.
3. `aws_iam_role_policy_attachment`.

Verification:

1. Get the policy ARN from the Terraform state.
2. Get the default policy version ID: `aws iam get-policy --policy-arn <policy-arn>`.
3. Get the policy document: `aws iam get-policy-version --policy-arn <policy-arn> --version-id <version-id>`.
4. Parse the policy document's JSON.
5. The script will iterate through all `Action` arrays in the policy's `Statement`.
6. It will fail if any action string is `"*"` or contains a `*` character, unless the string is one of the permitted CloudWatch Logs actions (e.g., `logs:CreateLogStream`, `logs:PutLogEvents`). The check must ensure S3 and DynamoDB permissions are specific (e.g., `s3:GetObject`, `dynamodb:PutItem`).

---

6. The Terraform configuration must define a Lambda function and configure an S3 bucket notification to trigger it upon object creation.

Contract Specification

partnr

1. `aws_lambda_function`
2. `aws_s3_bucket_notification`
3. `aws_lambda_permission`

Verification:

1. Get the function name and bucket name from the Terraform state.
2. **Check Lambda Function:** Run `aws lambda get-function --function-name <function-name>`. The command must succeed and return the function's configuration, including the attached IAM role.
3. **Check S3 Trigger:** Run `aws s3api get-bucket-notification-configuration --bucket <bucket-name>`. The response must contain a `LambdaFunctionConfigurations` array. This array must have an entry where the `LambdaFunctionArn` matches the created Lambda function's ARN and the `Events` array contains `s3:ObjectCreated:*`.

---

7. The complete workflow must be functional: uploading a file to the S3 bucket must trigger the Lambda function, which in turn writes a new item to the DynamoDB table.

Contract Specification

Verification:

1. Generate a unique test file (e.g., `test-event-$(uuidgen).json`).
2. Use `aws s3 cp` to upload the test file to the provisioned S3 bucket.
3. Wait for a reasonable period (e.g., 10 seconds) for the event to propagate.
4. Use `aws dynamodb scan --table-name <table-name>` to query the DynamoDB table.
5. The returned items must contain a record corresponding to the uploaded test file. The verifier will check for an item where the `ObjectKey` attribute matches the filename of the uploaded test file.

---

8. A shell script named `compliance_check.sh` must be present in the repository root. The script must be executable and must exit with a status code of 0, indicating all its internal checks have passed.

Contract Specification

File Location: `compliance_check.sh`

Verification:

1. Check for the existence of `compliance_check.sh` at the project root.
2. Set execute permissions: `chmod +x compliance_check.sh`.

3. Execute the script: `./compliance_check.sh dev`.

partnr

---

9. The Terraform project must be structured to support multiple environments using Terraform workspaces. The resource names should be parameterized by the workspace name.

   Contract Specification

   Terraform Configuration:

partnr

About Us

Contact Us

Privacy Policy

Terms and Conditions

All rights reserved. Copyright, Partnr 2025-26